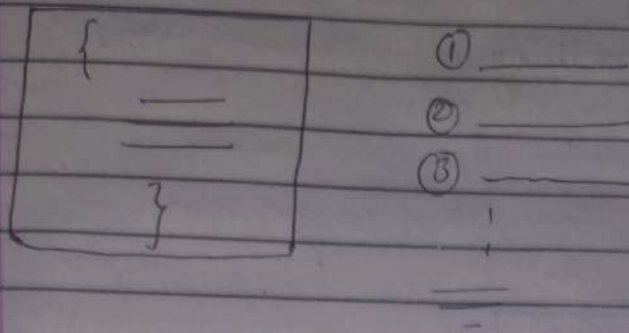


Flow of control

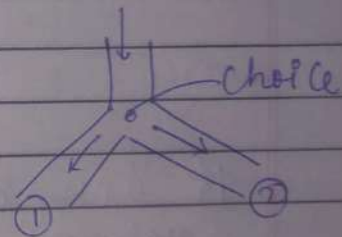


① Selection ② Iteration ③ Jumps

- ↓
- ① if ()
 - ② if-else ()
 - ③ switch

```

{
    int i = 4
    system.out.println(i);
    int j = 5;
}
    
```



Syntax of IF;

→ True/False. boolean

```
if (expression) {
    // do something
}
```

```
boolean b = true
```

```
if (b == true) {
    // do something
}
```

b = false

```
if ( ) {
    // do something
}
```

```
else {
    // do something
}
```

```
}
```

if inner expression is true then if statement will be executed, otherwise move forward.

Switch statements

↳ as like if-else-if ladder more optimized



```
if (choice == 1) {
    // do something
} else if (c == 2) {
    // do something
}
```

Syntax.

```
int i = 3;
```

```
switch (i) {
```

Case 1:

```
System.out.println("i is 1");
break;
```

Case 2:

```
System.out.println("i is 2");
break;
```

Case 3:

```
SOP ("i is 3");
break;
```

default:

```
SOP (" ");
break;
```

```
}
```


switch → limited use case
should evaluate to

switch (expression) → byte, short, int, char
enumeration.

No duplicate cases allowed.

→ After JDK 7, string can also be used as
switch expression.

→ After JDK-14, switch is enhanced too much.

Switch vs if-else-if ladder.

① Switch can only test equality but if-else
if can test both equality & inequality.

② Switch is more efficient than if-else-if ladder.

switch () {
 // check equality
 if () {
 // check both equality
 & inequality
 }
}

else-if {
}

else {
}

JVM internally may generate

① Jump table (if cases are continuous numbers)
② Lookup table (if cases are scattered)

Compiler & JVM automatically handle it.

Jump Table,

1 - case 1 { } → jump table is like array but
2 - case 2 { } not an array
3 - case 3 { } → it is a compiler optimization
 technique used internally
 when you write switch statements
default - default { }

or

0	1	2	3
case 1	case 2	case 3	default

Jump table;

It are not always efficient.

```

i = 3
Switch (i) {
    Case 1: -
    Case 2: -
    Case 3: -
    Case 4: -
}
    
```

} - spare value.

(dense)

Jump Table

Table switch look up - switch * (value are random)

0	1	2	3

1	1000	100000
0	1	2

i = 3000 Binary search concept.

Case 1
Case 2000
Case 3000
Case 10,000
Case 1001000

Switch Statement

Nested Switch.

```

Switch (i) {
    Switch (j) {
    }
}
    
```